Shravya Dsouza

Test ID: 432006969514075 | 9920180233 | 1032222324@mitwpu.edu.in

Test Date: September 28, 2024

Computer Science	Logical Ability	Computer Programming	Quantitative Ability (Advanced)
73 /100	Not completed	60 /100	53 /100

English Comprehension	WriteX - Essay Writing	Automata	Automata Fix
71 /100	76 /100	66 /100	15 /100

Personality

Completed

Computer Science

73 / 100

OS and Computer Architecture	DBMS	Computer Networks
82 / 100	67 / 100	56 / 100

Computer Programming

60 / 100

Basic Programming	Data Structures	OOP and Complexity Theory
59 / 100	62 / 100	60 / 100

Quantitative Ability (Advanced)

53 / 100

Basic Mathematics	Advanced Mathematics	Applied Mathematics
55 / 100	55 / 100	50 / 100

English Comprehension

71 / 100 | CEFR: C1

Grammar	Vocabulary	Comprehension
71 / 100	76 / 100	65 / 100

WriteX - Essay Writing

76 / 100 | CEFR: C1

Content Score	Grammar Score
79 / 100	68 / 100

Automata

66 / 100

Programming Ability	Programming Practices	Functional Correctness
60 / 100	100 / 100	61 / 100

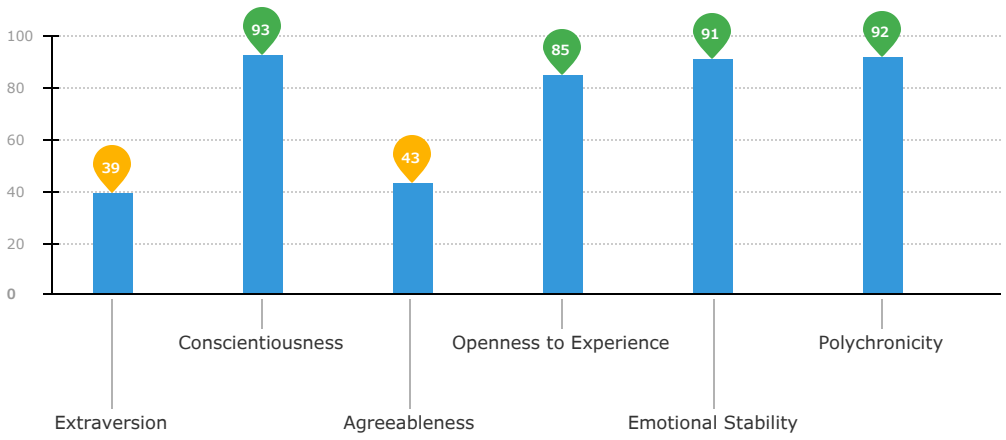
Automata Fix

15 / 100

Syntactical Error	Logical Error	Code Reuse
100 / 100	0 / 100	0 / 100

Personality

Completed



Competencies

People Interaction

Self-Drive

Trainability

Repetitive Job Suitability

Work attributes

1 | Introduction

About the Report

This report provides a detailed analysis of the candidate's performance on different assessments. The tests for this job role were decided based on job analysis, O*Net taxonomy mapping and/or criterion validity studies. The candidate's responses to these tests help construct a profile that reflects her/his likely performance level and achievement potential in the job role

This report has the following sections:

The **Summary** section provides an overall snapshot of the candidate's performance. It includes a graphical representation of the test scores and the subsection scores.

The **Insights** section provides detailed feedback on the candidate's performance in each of the tests. The descriptive feedback includes the competency definitions, the topics covered in the test, and a note on the level of the candidate's performance.

The **Response** section captures the response provided by the candidate. This section includes only those tests that require a subjective input from the candidate and are scored based on artificial intelligence and machine learning.

The **Learning Resources** section provides online and offline resources to improve the candidate's knowledge, abilities, and skills in the different areas on which s/he was evaluated.

Score Interpretation

All the test scores are on a scale of 0-100. All the tests except personality and behavioural evaluation provide absolute scores. The personality and behavioural tests provide a norm-referenced score and hence, are percentile scores. Throughout the report, the colour codes used are as follows:

- Scores between 67 and 100
- Scores between 33 and 67
- Scores between 0 and 33

2 | Insights

English Comprehension

71 / 100 | CEFR: **C1**

This test aims to measure your vocabulary, grammar and reading comprehension skills.

You have a fairly rich vocabulary and a strong command of English grammar. You are able to read and understand complex text. Having a good command of the English language is important to communicate with internal stakeholders and clients, as well as to interpret reports, articles and complex texts at work.

Quantitative Ability (Advanced)

53 / 100

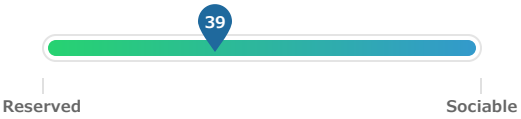
This test aims to measure your ability to solve problems on basic arithmetic operations, probability, permutations and combinations, and other advanced concepts.

You are able to solve word problems on basic concepts of percentages, ratio, proportion, interest, time and work. Having a strong hold on these concepts can help you understand the concept of work efficiency and how interest is accrued on bank savings. It can also guide you in time management, work planning, and resource allocation in complex projects.

Personality

Competencies

Extraversion



Extraversion refers to a person's inclination to prefer social interaction over spending time alone. Individuals with high levels of extraversion are perceived to be outgoing, warm and socially confident.

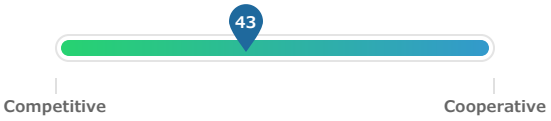
- You are comfortable socializing to a certain extent. You prefer small gatherings in familiar environments.
- You feel at ease interacting with your close friends but may be reserved among strangers.
- You indulge in activities involving thrill and excitement that are not too risky.
- You contemplate the consequences before expressing any opinion or taking an action.
- You take charge when the situation calls for it and you are comfortable following instructions as well.
- Your personality may be suitable for jobs demanding flexibility in terms of working well with a team as well as individually.

Conscientiousness

Conscientiousness is the tendency to be organized, hard working and responsible in one's approach to your work. Individuals with high levels of this personality trait are more likely to be ambitious and tend to be goal-oriented and focused.

- You value order and self discipline and tends to pursue ambitious endeavours.
- You believe in the importance of structure and is very well-organized.
- You carefully review facts before arriving at conclusions or making decisions based on them.
- You strictly adhere to rules and carefully consider the situation before making decisions.
- You tend to have a high level of self confidence and do not doubt your abilities.
- You generally set and work toward goals, try to exceed expectations and are likely to excel in most jobs, especially those which require careful or meticulous approach.

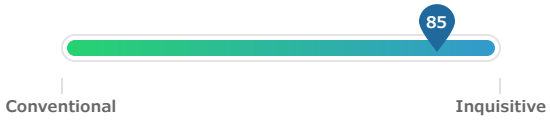
Agreeableness



Agreeableness refers to an individual's tendency to be cooperative with others and it defines your approach to interpersonal relationships. People with high levels of this personality trait tend to be more considerate of people around them and are more likely to work effectively in a team.

- You are flexible regarding your opinions and be willing to accommodate the needs of others.
- You are generally considerate of the needs of others yet may, at times, overlook social norms to achieve personal success.
- You are selective about the people you choose to trust.
- You are caring and you empathise a friend in distress.
- You give credit to others but also tends to be open with your friends about personal achievements.
- You are more inclined to strike a compromise in tough situations and may be suitable for jobs that demand managing expectations among different stakeholders.

Openness to Experience

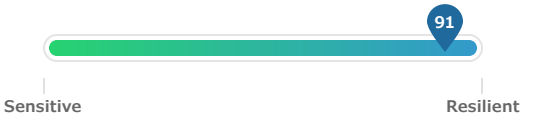


Openness to experience refers to a person's inclination to explore beyond conventional boundaries in different aspects of life. Individuals with high levels of this personality trait tend to be more curious, creative and innovative in nature.

- You tend to be curious in nature and is generally open to trying new things outside your comfort zone.
- You may have a different approach to solving conventional problems and tend to experiment with those solutions.
- You are creative and tends to appreciate different forms of art.
- You are likely to be in touch with your emotions and is quite expressive.

- Your personality is more suited for jobs requiring creativity and an innovative approach to problem solving.

Emotional Stability



Emotional stability refers to the ability to withstand stress, handle adversity, and remain calm and composed when working through challenging situations. People with high levels of this personality trait tend to be more in control of their emotions and are likely to perform consistently despite difficult or unfavourable conditions.

- You are calm and composed in nature.
- You tend to maintain composure during high pressure situations.
- You are very confident and comfortable being yourself.
- You find it easy to resist temptations and practice moderation.
- You are likely to remain emotionally stable in jobs with high stress levels.

Polychronicity

Polychronicity refers to a person's inclination to multitask. It is the extent to which the person prefers to engage in more than one task at a time and believes that such an approach is highly productive. While this trait describes the personality disposition of a person to multitask, it does not gauge their ability to do so successfully.

- You pursue multiple tasks simultaneously, switching between them when needed.
- You prefer working to achieve some progress on multiple tasks simultaneously than completing one task before moving on to the next task.
- You tend to believe that multitasking is an efficient way of doing things and prefers an action packed work life with multiple projects.

3 | Response

Automata

66 / 100 Code Replay

Question 1 (Language: Python3)

A company is planning a big sale where they will give their customers a special promotional discount. Each customer that purchases a product from the company has a unique customerID numbered from 0 to N-1. Andy, the head of marketing, has selected the total bills of the N customers for the promotional scheme. The discount will be given to the customers whose total bills are perfect squares. The customers may use this discount on a future purchase.

Write an algorithm to help Andy find the number of customers that will be given discounts.

Scores

Programming Ability

100 / 100

Completely correct. A correct implementation of the problem using the right control-structures and data dependencies.

Programming Practices

100 / 100

High readability, high on program structure. The source code is readable and does not consist of any significant redundant/improper coding constructs.

Functional Correctness

100 / 100

Functionally correct source code. Passes all the test cases in the test suite for a given problem.

Final Code Submitted — **Compilation Status: Pass**

```
1
2 """
3 bill, representing the bill amounts
4 """
5 import math
6
7 def countCustomers(bill,bill_size):
8     t = 0
9     # Write your code here
10    for i in range(bill_size):
11        if math.sqrt(bill[i]) == int(math.sqrt(bill[i])):
12            t = t+1
13        else:
14            continue
15
16    return (t)
```

Code Analysis

Average-case Time Complexity

Candidate code: O(N)

Best case code: O(N)

*N represents number of customers.

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

```
17
18 def main():
19     #input for bill
20     bill = []
21     bill_size  = int(input())
22     bill = list(map(int,input().split()))
23
24
25     result = countCustomers(bill,bill_size)
26     print(result)
27
28 if __name__ == "__main__":
29     main()
30
31
```

Readability & Language Best Practices

Line 25: too many blank lines (2)
Line 16: Unnecessary parens after 'return' keyword
Line 8,12: Variable name doesn't conform to snake_case naming style

Test Case Execution

Passed TC: **100%**

Total score

13/13

100%	100%	100%
Basic(**6**/6)	Advance(**6**/6)	Edge(**1**/1)

Compilation Statistics

22	21	1	0	0	3
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:28:00**
Average time taken between two compile attempts:	**00:01:16**
Average test case pass percentage per compile:	**5.59%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: Python3)

The computer systems of N employees of a company are arranged in a row. A technical fault in the power supply has caused some of the systems to turn OFF while the others remain ON. The employees whose systems are OFF are unable to work. The company does not like to see its employees sitting idle. So until the technical team can find the actual cause of the breakdown, the technical head has devised a temporary workaround for the OFF systems at a minimum cost. They decide to connect all the OFF systems to the nearest ON system with the shortest possible length of cable. To make this happen, they calculate the distance of each system from the first system.

Write an algorithm to help the technical head find the minimum length of cable they need to turn all the systems ON.

Scores

Programming Ability

20 / 100

Code seems to be unrelated to the given problem.

Programming Practices

100 / 100

High readability, high on program structure. The source code is readable and does not consist of any significant redundant/improper coding constructs.

Functional Correctness

22 / 100

Partially correct basic functionality. The source code compiles and passes only some of the basic test cases. Some advanced or edge cases may randomly pass.

Final Code Submitted

Compilation Status: Pass

```
1
2  """
3
4  """
5  def minLength(systemState, dist,systemState_size):
6      # Write your code here
7      t = 0
8      for i in range(systemState_size):
9          if systemState[i] == 1:
10             continue
11         else:
12             print(dist[i])
13     return t
14
15 def main():
16     #input for systemState
17     systemState = []
18     systemState_size  = int(input())
19     systemState = list(map(int,input().split()))
20
21     #input for dist
22     dist = []
23     dist_size  = int(input())
24     dist = list(map(int,input().split()))
25
26
27     result = minLength(systemState, dist,systemState_size)
28     print(result)
29
30 if __name__ == "__main__":
31     main()
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code: O(N)

*N represents number of systems

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

Readability & Language Best Practices

Line 27: too many blank lines (2)
Line 5,7: Argument name "systemState" doesn't conform to snake_case naming style

Test Case Execution

Passed TC: **9.52%**

Total score

2/21

13%	0%	25%
Basic(**1**/8)	Advance(**0**/9)	Edge(**1**/4)

Compilation Statistics

7	7	0	0	0	5
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:15:29**
Average time taken between two compile attempts:	**00:02:13**
Average test case pass percentage per compile:	**1.36%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Automata Fix

15 / 100 | Code Replay

Question 1 (Language: C++)

The function/method ***multiplyNumber*** returns an integer representing the multiplicative product of the maximum two of three input numbers. The function/method ***multiplyNumber*** accepts three integers- *numA*, *numB* and *numC*, representing the input numbers.

The function/method ***multiplyNumber*** compiles unsuccessfully due to syntactical error. Your task is to debug the code so that it passes all the test cases.

Scores

Final Code Submitted

Compilation Status: Pass

```
1  // You can print the values to stdout for debugging
2  using namespace std;
3  int multiplyNumber(int numA, int numB, int numC)
4  {
5    int result,min,max,mid;
6    max=(numA>numB)?((numA>numC)?numA:numC):((numB>numC)?numB:numC);
7    min=(numA<numB)?((numA<numC)?numA:numC):((numB<numC)?numB:numC);
8    mid=(numA+numB+numC)-(min+max);
9    result=(max* int(mid));
10   return result;
11 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **100%**

Total score

10/10

100%	100%	0%
Basic(**7**/7)	Advance(**3**/3)	Edge(**0**/0)

Compilation Statistics

5	1	4	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:05:07**
Average time taken between two compile attempts:	**00:01:01**
Average test case pass percentage per compile:	**20%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 2 (Language: C++)

The function/method ***sameElementCount*** returns an integer representing the number of elements of the input list which are even numbers and equal to the element to its right. For example, if the input list is [4 4 4 1 8 4 1 1 2 2] then the function/method should return the output '3' as it has three similar groups i.e, (4, 4), (4, 4), (2, 2).

The function/method ***sameElementCount*** accepts two arguments - *size*, an integer representing the size of the input list and *inputList*, a list of integers representing the input list.

The function/method compiles successfully but fails to return the desired result for some test cases due to incorrect implementation of the function/method ***sameElementCount***. Your task is to fix the code so that it passes all the test cases.

Note:
In a list, an element at index i is considered to be on the left of index i+1 and to the right of index i-1. The last element of the input list does not have any element next to it which makes it incapable to satisfy the second condition and hence should not be counted.

Scores

Final Code Submitted **Compilation Status: Pass**

```
1 // You can print the values to stdout for debugging
2 using namespace std;
3 int sameElementCount(int size, int* inputList)
4 {
5    int i,count =0;
6    for(i=0;i<size-1;i++)
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

```
7   {
8     if((inputList[i]%2==0)&&(inputList[i]==inputList[i++])&&(inputLis
    t[i++]%2==0))
9       count++;
10  }
11  return count;
12 }
13
```

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **87.5%**

Total score

7/8

86%	0%	100%
Basic(**6**/7)	Advance(**0**/0)	Edge(**1**/1)

Compilation Statistics

2	2	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:02:46**
Average time taken between two compile attempts:	**00:01:23**
Average test case pass percentage per compile:	**50%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 3 (Language: C++)

The function/method ***countOccurrence*** return an integer representing the count of occurrences of given value in the input list.
The function/method ***countOccurrence*** accepts three arguments - *len*, an integer representing the size of the input list, *value,* an integer representing the given value and *arr*, a list of integers, representing the input list.

The function/method ***countOccurrence*** compiles successfully but fails to return the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted **Compilation Status: Pass**

```
1  // You can print the values to stdout for debugging
2  int countOccurrence(int len, int value, int *arr)
3  {
4     int i=0, count = 0;
5     while(i<len)
6     {
7        if(arr[i]==value)
8           count += 1;
9     }
10    return count;
11 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **11.11%**

Total score

1/9

0%	0%	100%
Basic(**0**/3)	Advance(**0**/5)	Edge(**1**/1)

Compilation Statistics

5	3	2	3	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:03:53**
Average time taken between two compile attempts:	**00:00:47**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 4 (Language: Java)

You are given a predefined structure/class ***Point*** and also a collection of related functions/methods that can be used to perform some basic operations on the structure.

The function/method ***isRightTriangle*** returns an integer '1', if the points make a right-angled triangle otherwise return

'0'.
The function/method ***isRightTriangle*** accepts three points - *P1, P2, P3* representing the input points.

You are supposed to use the given function to complete the code of the function/method ***isRightTriangle*** so that it passes all test cases.

Helper Description
The following class is used to represent a point and is already implemented in the default code (Do not write this definition again in your code):

```
public class Point
{
   public int x, y;
   public Point()
   { }
   public Point(int x1, int y1)
   {
      x = x1 ;
      y = y1 ;
   }
   public double calculateDistance(Point P)
   {
      /*Return the euclidean distance between two input points.
        This can be called as -
        *  If P1 and P2 are two points then -
        *  P1.calculateDistance(P2);*/
   }
}
```

Scores

The candidate did not make any changes in the code.

Compilation Statistics

0	0	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:00**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 5 (Language: C++)

The function/method ***manchester*** print space-separated integers with the following property: for each element in the input list *arr*, if the bit arr[i] is the same as *arr*[i-1], then the element of the output list is 0. If they are different, then its 1. For the first bit in the input list, assume its previous bit to be 0. This encoding is stored in a new list.

The function/method ***manchester*** accepts two arguments - *len,* an integer representing the length of the list and *arr* and *arr*, a list of integers, respectively. Each element of *arr* represents a bit - 0 or 1

For example - if *arr* is {0 1 0 0 1 1 1 0}, the function/method should print an list {0 1 1 0 1 0 0 1}.

The function/method compiles successfully but fails to print the desired result for some test cases due to logical errors. Your task is to fix the code so that it passes all the test cases.

Scores

Final Code Submitted — Compilation Status: Pass

```
1  // You can print the values to stdout for debugging
2  void manchester(int len, int* arr)
3  {
4     int *res = new int[len];
5     res[0] = arr[0];
6     for(int i = 1; i < len; i++){
7       res[i] = (arr[i]==arr[i-1]);
8     }
9     for(int i=0; i<len; i++)
10       printf("%d ", res[i]);
11 }
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution — Passed TC: **20%**

Total score 1/5

50%	0%	0%
Basic(**1**/2)	Advance(**0**/3)	Edge(**0**/0)

Compilation Statistics

0	0	0	1	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:18**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**20%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 6 (Language: Java)

The function/method ***drawPrintPattern*** accepts *num*, an integer.
The function/method ***drawPrintPattern*** prints the first *num* lines of the pattern shown below.

For example, if *num* = 3, the pattern should be:

```
1 1
1 1 1 1
1 1 1 1 1 1
```

The function/method ***drawPrintPattern*** compiles successfully but fails to get the desired result for some test cases due to incorrect implementation of the function/method. Your task is to fix the code so that it passes all the test cases.

Scores

The candidate did not make any changes in the code.

Compilation Statistics

0	0	0	0	0	0
Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors

Response time:	**00:00:00**
Average time taken between two compile attempts:	**00:00:00**
Average test case pass percentage per compile:	**0%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

Question 7 (Language: C++)

The function/method ***median*** accepts two arguments - *size* and *inputList*, an integer representing the length of a list and a list of integers, respectively.

The function/method ***median*** is supposed to calculate and return an integer representing the median of elements in the input list. However, the function/method **median** works only for odd-length lists because of incomplete code.

You must complete the code to make it work for even-length lists as well. A couple of other functions/methods are available, which you are supposed to use inside the function/method **median** to complete the code.

Helper Description

The following function is used to represent a quick_select and is already implemented in the default code (Do not write this definition again in your code):

```
int quick_select(int* inputList, int start_index, int end_index, int median_order)
{
  /*It calculate the median value
  This can be called as -
  quick_select(inputList, start_index, end_index, median_order)
  where median_order is the half length of the inputList
}
```

Scores

Final Code Submitted

Compilation Status: Pass

```
1  // You can print the values to stdout for debugging
2  using namespace std;
3  float median(int size, int* inputList)
4  {
5    int start_index = 0;
6    int end_index = size-1;
7    float res = -1;
8    if(size%2!=0) // odd length arrays
9    {
10     int median_order = ((size+1)/2);
11     res = (float)quick_select(inputList, start_index, end_index, median_order);
12   }
13   else // even length arrays
14   {
15     int term = ((size+1)/2) + int(size/2) ;
16     res = (float)quick_select(inputList, start_index, end_index, term);
17
18   }
19   return res;
20 }
21
22
```

Code Analysis

Average-case Time Complexity

Candidate code: Complexity is reported only when the code is correct and it passes all the basic and advanced test cases.

Best case code:

*N represents

Errors/Warnings

There are no errors in the candidate's code.

Structural Vulnerabilites and Errors

There are no errors in the candidate's code.

Test Case Execution

Passed TC: **42.86%**

Total score

3/7

50%	0%	100%
Basic(**2**/4)	Advance(**0**/2)	Edge(**1**/1)

Compilation Statistics

Total attempts	Successful	Compilation errors	Sample failed	Timed out	Runtime errors
3	3	0	3	0	0

Response time:	**00:07:27**
Average time taken between two compile attempts:	**00:02:29**
Average test case pass percentage per compile:	**14.3%**

Average-case Time Complexity

Average Case Time Complexity is the order of performance of the algorithm given a random set of inputs. This complexity is measured here using the Big-O asymptotic notation. This is the complexity detected by empirically fitting a curve to the run-time for different input sizes to the given code. It has been benchmarked across problems.

Test Case Execution

There are three types of test-cases for every coding problem:

Basic: The basic test-cases demonstrate the primary logic of the problem. They include the most common and obvious cases that an average candidate would consider while coding. They do not include those cases that need extra checks to be placed in the logic.

Advanced: The advanced test-cases contain pathological input conditions that would attempt to break the codes which have incorrect/semi-correct implementations of the correct logic or incorrect/semi-correct formulation of the logic.

Edge: The edge test-cases specifically confirm whether the code runs successfully even under extreme conditions of the domain of inputs and that all possible cases are covered by the code

WriteX - Essay Writing

76 / 100 | CEFR: **C1**

Question

In your opinion, when is it justified for a company to fire its employees from the job? What are the considerations to be taken before making such a decision?
Support your response with reasons and suitable examples.

Scores

Content Score	Grammar Score
79 / 100	**68** / 100

Response | **Error Summary**

No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following decisions are made to minimize the consequences" , Ratan Tata , Leader of the Tata Group. The quotation may not be the exact words , however , you as the reader understand the reasoning behind it. Recently , Elon Musk made the top trending headline saying , "Eon Musk fires 300 employees from the company , who have served the company for many years, in the online Zoom meet". This article spread across the globe like a deadly virus and became one of the biggest debates on news channels. As a boss or an authorative figure , responsible for the team to deliever the desired result that aligns with the company's goal , it is a skill to encourage and guide the team members to play to their strengths ,foster healthy communication and accept valid criticism. Firing of employees , especially the ones who have served in the company for a longer tenure , has always been seen in the bad light. The decisions are tough to make , owing to the emotional aspect , but a required one when the employee fails to meet the expected results . Keeping in mind , these results are practical and feasible with justified period of completion. It is mandatory for the employee to take the accountability for the completion of the given task. Failing to meet the deadline must be justified with valid reasons. As a company , that discourages toxic work cultures, must rationally take decisions and effectively communicate with the employee. Just like in our school days , when we used to receive three warnings for misbehaviour before the suspension , the same analogy , in my opinion , must be delievered through formal emails to the employee. I strongly believe in second chance , however , "once bitten , twice alert" stays true .The company can fire an employee when the employee fails to improve despite the warnings. Accountability being the key factor to the decision making , the employee must respond to the emails with valid reasons and progressively strive to show improvements in the respective area. The employee must be punctual and

Spelling	8
White Space	33
Style	0
Grammar	22
Typographical	6

Essay Statistics

387	16	24	211	143
Total words	Total sentences	Average sentence length	Total unique words	Total stop words

Error Details

Spelling

...e made to minimize the consequences" , Ratan Tata , Leader of the Tata Group. The qu...	Possible spelling mistake found
... to minimize the consequences" , Ratan Tata , Leader of the Tata Group. The quotati...	Possible spelling mistake found
...equences" , Ratan Tata , Leader of the Tata Group. The quotation may not be the exa...	Possible spelling mistake found
...nd the reasoning behind it. Recently , Elon Musk made the top trending headline say...	Possible spelling mistake found
...bates on news channels. As a boss or an authorative figure , responsible for the team to de...	Possible spelling mistake found

...ve figure , responsible for the team to deliever the desire d result that aligns with the...	Possible spelling mistake found
...te with the employee. Just like in our school days , when we used to receive three warning...	This expression is normally spelled as one or with hyphen.
... same analogy , in my opinion , must be delievered thro ugh formal emails to the employee. ...	Possible spelling mistake found
...ountability being the key factor to the decision making , the employee must respond to the ema...	This word is normally spelled with hyphen.

White Space

No Decision is Right "No Decision is Right , if the Decision ...	Possible typo: you repeated a whitespace
...Decision is Right "No Decision is Right , if the Decision pr oduces a successful o...	Put a space after the comma, but not before the comma
...e Decision produces a successful outcome , it is termed a s Successful Decision, el...	Put a space after the comma, but not before the comma
..., else if the outcome leads to a failure , following decisio ns are made to minimiz...	Put a space after the comma, but not before the comma
... are made to minimize the consequences" , Ratan Tata , Leader of the Tata Group. ...	Put a space after the comma, but not before the comma
...minimize the consequences" , Ratan Tata , Leader of the Tata Group. The quotation...	Put a space after the comma, but not before the comma
...The quotation may not be the exact words , however , yo u as the reader understand ...	Put a space after the comma, but not before the comma
...ion may not be the exact words , however , you as the r eader understand the reason...	Put a space after the comma, but not before the comma
...stand the reasoning behind it. Recently , Elon Musk mad e the top trending headlin...	Put a space after the comma, but not before the comma
...sk made the top trending headline saying , "Eon Musk fir es 300 employees from the ...	Put a space after the comma, but not before the comma
...usk fires 300 employees from the company , who have s erved the company for many ye...	Put a space after the comma, but not before the comma
...nels. As a boss or an authorative figure , responsible for the team to deliever th...	Put a space after the comma, but not before the comma
...sult that aligns with the company's goal , it is a skill to e ncourage and guide th...	Put a space after the comma, but not before the comma
... team members to play to their strengths ,foster healthy communication and accept ...	Put a space after the comma, but not before the comma
...mmunication and accept valid criticism. Firing of employ ees , especially the one...	Possible typo: you repeated a whitespace
...t valid criticism. Firing of employees , especially the ones who have served in ...	Put a space after the comma, but not before the comma
...erved in the company for a longer tenure , has always be en seen in the bad light. ...	Put a space after the comma, but not before the comma

...d light. The decisions are tough to make , owing to the e motional aspect , but a r...	Put a space after the comma, but not before the comma
... to make , owing to the emotional aspect , but a require d one when the employee fa...	Put a space after the comma, but not before the comma
...loyee fails to meet the expected results . Keeping in min d , these results are pra...	Don't put a space before the full stop
...t the expected results . Keeping in mind , these results a re practical and feasibl...	Put a space after the comma, but not before the comma
...stified with valid reasons. As a company , that discourag es toxic work cultures, m...	Put a space after the comma, but not before the comma
... employee. Just like in our school days , when we used t o receive three warnings ...	Put a space after the comma, but not before the comma
...s for misbehaviour before the suspension , the same ana logy , in my opinion , must...	Put a space after the comma, but not before the comma
...before the suspension , the same analogy , in my opinio n , must be delievered thro...	Put a space after the comma, but not before the comma
...nsion , the same analogy , in my opinion , must be delie vered through formal email...	Put a space after the comma, but not before the comma
...yee. I strongly believe in second chance , however , "onc e bitten , twice alert" s...	Put a space after the comma, but not before the comma
...ongly believe in second chance , however , "once bitten , twice alert" stays true ...	Put a space after the comma, but not before the comma
...n second chance , however , "once bitten , twice alert" st ays true .The company ca...	Put a space after the comma, but not before the comma
..., "once bitten , twice alert" stays true .The company can fire an employee when th...	Don't put a space before the full stop
..."once bitten , twice alert" stays true .The company can fi re an employee when the e...	Add a space between sentences
...ng the key factor to the decision making , the employee must respond to the email...	Put a space after the comma, but not before the comma
... the key factor to the decision making , the employee m ust respond to the emails ...	Possible typo: you repeated a whitespace

Grammar

No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following d ecisions are made to minimize the consequences" , Ratan T ata , Leader of the Tata Group.	Possible grammar error found. Consider replacing it with "Right.".
No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following d ecisions are made to minimize the consequences" , Ratan T ata , Leader of the Tata Group.	Possible grammar error found. Consider inserting "a" over here.

No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following d ecisions are made to minimize the consequences" , Ratan T ata , Leader of the Tata Group.	Possible grammar error found. Consider replacing it with "failure,".
No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following d ecisions are made to minimize the consequences" , Ratan T ata , Leader of the Tata Group.	Possible grammar error found. Consider replacing it with "consequences".
No Decision is Right "No Decision is Right , if the Decision produces a successful outcome , it is termed as Successful Decision, else if the outcome leads to a failure , following d ecisions are made to minimize the consequences" , Ratan T ata , Leader of the Tata Group.	Possible grammar error found. Consider removing "Group." from here.
The quotation may not be the exact words , however , you as the reader understand the reasoning behind it.	Possible grammar error found. Consider replacing it with "reader,".
Recently , Elon Musk made the top trending headline sayin g , "Eon Musk fires 300 employees from the company , wh o have served the company for many years, in the online Z oom meet".	Possible grammar error found. Consider replacing it with "meet.".
As a boss or an authorative figure , responsible for the tea m to deliever the desired result that aligns with the compa ny's goal , it is a skill to encourage and guide the team me mbers to play to their strengths ,foster healthy communica tion and accept valid criticism.	Possible grammar error found. Consider replacing it with "delivering".
As a boss or an authorative figure , responsible for the tea m to deliever the desired result that aligns with the compa ny's goal , it is a skill to encourage and guide the team me mbers to play to their strengths ,foster healthy communica tion and accept valid criticism.	Possible grammar error found. Consider inserting "is" over here.
Firing of employees , especially the ones who have served i n the company for a longer tenure , has always been seen in the bad light.	Possible grammar error found. Consider inserting "The firing" over here.
Firing of employees , especially the ones who have served i n the company for a longer tenure , has always been seen in the bad light.	Possible grammar error found. Consider replacing it with "a".
The decisions are tough to make , owing to the emotional aspect , but a required one when the employee fails to me et the expected results .	Possible grammar error found. Consider replacing it with "achieve".
Keeping in mind , these results are practical and feasible w ith justified period of completion.	Possible grammar error found. Consider replacing it with "Keep".
Keeping in mind , these results are practical and feasible w ith justified period of completion.	Possible grammar error found. Consider inserting "a" over here.
Failing to meet the deadline must be justified with valid rea sons.	Possible grammar error found. Consider replacing it with "by".
As a company , that discourages toxic work cultures, must rationally take decisions and effectively communicate with the employee.	Possible grammar error found. Consider inserting "they" over here.

As a company , that discourages toxic work cultures, must rationally take decisions and effectively communicate with the employee.	Possible grammar error found. Consider replacing it with "employee.ssssss".
Just like in our school days , when we used to receive thre e warnings for misbehaviour before the suspension , the sa me analogy , in my opinion , must be delievered through f ormal emails to the employee.	Possible grammar error found. Consider replacing it with "about".
I strongly believe in second chance , however , "once bitte n , twice alert" stays true .	Possible grammar error found. Consider replacing it with "chance.".
Accountability being the key factor to the decision making , the employee must respond to the emails with valid reason s and progressively strive to show improvements in the res pective area.	Possible grammar error found. Consider replacing it with "in".
Accountability being the key factor to the decision making , the employee must respond to the emails with valid reason s and progressively strive to show improvements in the res pective area.	Possible grammar error found. Consider removing "the" from here.
Accountability being the key factor to the decision making , the employee must respond to the emails with valid reason s and progressively strive to show improvements in the res pective area.	Possible grammar error found. Consider replacing it with "for".

Typographical

No Decision is Right "No Decision is Right , if the Decision p...	Use a smart opening quote here: "“".
...ns are made to minimize the consequences" , Ratan Tata , Leader of the Tata Grou...	Use a smart closing quote here: "”".
...made the top trending headline saying , "Eon Musk fires 300 employees from the co...	Use a smart opening quote here: "“".
... for many years, in the online Zoom meet". This article s pread across the globe l...	Use a smart closing quote here: "”".
...ly believe in second chance , however , "once bitten , twi ce alert" stays true .T...	Use a smart opening quote here: "“".
...e , however , "once bitten , twice alert" stays true .The c ompany can fire an emp...	Use a smart closing quote here: "”".

4 | Learning Resources

English Comprehension

Read the latest articles by The Economist	
Read novels to enhance your comprehension skills	
Read opinions to improve your comprehension	

Quantitative Ability (Advanced)

Watch a video on the history of algebra and its applications	
Learn about proportions and its practical usage	
Learn about calculating percentages manually	

Icon Index

 Free Tutorial

 Paid Tutorial

 Youtube Video

 Web Source

 Wikipedia

 Text Tutorial

 Video Tutorial

 Google Playstore